CareerLens — Offline AI Resume & Interview Coach

IEEE Computer Society Bangalore Chapter · Girl Geeks 2026 · Official Use Case 01

CareerLens is a private, offline interview coach that questions you on your own resume. Everything runs on the student's laptop: the language model, speech recognition and webcam analysis. No data is sent to the internet.

What it does:

1. **Resume analysis** — reads a PDF resume, runs 10 ATS checks, finds missing keywords for the target role, and rewrites weak bullet points.
2. **Mock interview** — 5 questions: an introduction, 2 questions about specific claims on the student's own resume, 1 technical question for the role, and 1 behavioural question. Answers can be spoken or typed.
3. **Feedback** — 4 scores (Communication, Confidence, Technical, Resume), feedback on every answer with a stronger sample answer, and a 2-week learning plan with free resources.
4. **GazeGuard-lite** — the webcam measures eye contact and look-aways. Video never leaves the browser.
5. **CopilotShield-lite** — detects hidden AI-assistant windows (the trick used by tools like Parakeet AI) and known copilot apps, shown as a live status light. Windows only.

Roles supported: **SDE, Data Analyst, ML Engineer, Embedded, Web Developer**.

Contents

1. Folder structure
2. What to install first
3. Backend setup
4. Frontend setup
5. One-time downloads (needs internet)
6. Test each part
7. Run CareerLens
8. Using the app
9. Demo day checklist
10. Settings you can change
11. API reference
12. Troubleshooting
13. Team ownership
14. Privacy

Commands are given for **Windows PowerShell** and **Linux/macOS**. On Windows type `python`; on Linux/macOS type `python3`.

---

1. Folder structure

```
careerlens/
├── README.md                    this file
├── CHANGES.md                   fixes made during the integration check
├── start-careerlens.bat         Windows: starts everything with one double-
click / command
├── start-careerlens.sh          Linux/macOS: starts everything with one
command
├── backend/                     Python FastAPI server (port 8000)
|   ├── main.py                  all API routes (Member 5)
|   ├── llm.py                   shared gateway to Ollama (Member 5)
|   ├── db.py                    SQLite history (Member 5)
|   ├── shield.py                CopilotShield-lite (Member 5)
|   ├── test_overlay.py          demo window hidden from screen-share
(Member 5)
|   ├── prewarm.py               run before the demo (Member 5)
|   ├── resume.py                resume analysis (Member 2)
|   ├── scoring.py               final scores + roadmap (Member 2)
|   ├── interview.py             questions, Whisper, answer grading (Member
3)
|   ├── gaze.py                  checks gaze numbers (Member 4)
|   ├── requirements.txt
|   ├── data/
|   |   ├── role_keywords.json   keywords per role (Member 2)
|   |   ├── resources.json       free learning links (Member 2)
|   |   ├── question_bank.json   technical + behavioural questions (Member
3)
|   |   ├── known_tools.json     copilot app names + allow-list (Member 5)
|   |   ├── samples/sample_resume.pdf
|   |   └── careerlens.db        created automatically
|   └── mocks/                   example API responses
└── frontend/                    React + Vite web app (port 5173)
    ├── package.json
    ├── .env.example
    ├── public/mediapipe/        face-tracking files (you add these in step
4)
    └── src/
        ├── api.js               every backend call
        ├── pages/               Upload, ResumeReport, InterviewRoom,
FinalReport, History
        ├── components/          score ring, bars, answer cards, shield
light...
```

```
        ├── hooks/useRecorder.js  microphone recording
        └── gaze/                 GazeGuard-lite (Member 4)
```

2. What to install first

Install these once on every laptop that will run CareerLens.

Tool	Version	Download	Notes
Python	3.11 (3.10–3.12 work)	https://www.python.org/downloads/	Windows: tick **"Add python.exe to PATH"** on the first installer screen
Node.js	20 LTS or newer	https://nodejs.org/	Choose the LTS installer
Git	any	https://git-scm.com/downloads	Only needed to clone the repo
Ollama	latest	https://ollama.com/download	Runs the local language model
Browser	Chrome or Edge	—	Best webcam and microphone support

Hardware: 8 GB RAM minimum (16 GB recommended), about 6 GB free disk space, a webcam and a microphone. CopilotShield needs **Windows 10 version 2004 or newer, or Windows 11**.

Check everything installed. Close and reopen the terminal first, then run:

```powershell
python --version
node --version
npm --version
git --version
ollama --version
```

Each should print a version number. On Linux/macOS use `python3 --version`.

Windows only — allow PowerShell scripts (needed for `venv` and `npm`). Run once:

```powershell
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Type `Y` and press Enter if asked.

3. Backend setup

3.1 Get the code

From GitHub:

```powershell
git clone <your-repo-url> careerlens
cd careerlens
```

Or, if you have the zip: unzip it, then `cd` into the `careerlens` folder.

3.2 Download the language models

Ollama starts automatically after installing on Windows and macOS. On Linux, if `ollama list` gives an error, run `ollama serve` in a separate terminal first.

```powershell
ollama pull qwen2.5:3b
ollama pull qwen2.5:1.5b
ollama run qwen2.5:3b "Say hello in one line"
```

The last command should print a short greeting. `qwen2.5:3b` is about 2 GB; `qwen2.5:1.5b` is a smaller backup for weak laptops.

3.3 Create the Python environment and install packages

Windows PowerShell:

```powershell
cd backend
python -m venv venv
venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Linux/macOS:

```bash
```

```
cd backend
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

When the environment is active, the prompt starts with `(venv)`. **Every backend command in this README assumes you are inside `backend` with `(venv)` showing.** To activate it again later, repeat only the `activate` line.

---

4. Frontend setup

Open a **second terminal** in the `careerlens` folder.

Windows PowerShell:

```powershell
cd frontend
npm install
New-Item -ItemType Directory -Force public\mediapipe
Copy-Item -Recurse -Force node_modules\@mediapipe\tasks-vision\wasm public\medi
curl.exe -L -o public\mediapipe\face_landmarker.task https://storage.googleapis
```

Linux/macOS:

```bash
cd frontend
npm install
mkdir -p public/mediapipe
cp -r node_modules/@mediapipe/tasks-vision/wasm public/mediapipe/wasm
curl -L -o public/mediapipe/face_landmarker.task https://storage.googleapis.com
```

Check the face-tracking files are in place:

```powershell
dir public\mediapipe
dir public\mediapipe\wasm
```

(Linux/macOS: `ls -lh public/mediapipe public/mediapipe/wasm`.) You should see `face_landmarker.task` (about 3–4 MB) and a `wasm` folder containing

`vision_wasm_internal.js` and `vision_wasm_internal.wasm`.

Commit `public/mediapipe` to Git so the other laptops get it without downloading.

---

5. One-time downloads (needs internet)

CareerLens works offline **after** these files are on the laptop. Do this with Wi-Fi on, on every laptop you will demo with.

What	How it gets downloaded	Done in
Language models	`ollama pull ...`	step 3.2
Python packages	`pip install -r requirements.txt`	step 3.3
Frontend packages	`npm install`	step 4
Face-tracking model	`curl ... face_landmarker.task`	step 4
Whisper speech model (~150 MB)	first load, command below	now

Download the Whisper model now (in `backend`, with `(venv)` active):

```powershell
python -c "import interview; print('Whisper ready:', interview.whisper_ready())"
```

Expect `Whisper ready: True` after up to a minute. After this, it loads from the laptop's cache with no internet.

---

6. Test each part

Run these in `backend` with `(venv)` active. Each one tests one file on its own.

```powershell
python llm.py
```

Expect `Ollama running: True` and three interview tips as JSON.

```powershell
python resume.py data\samples\sample_resume.pdf SDE
```

(Linux/macOS: `data/samples/sample_resume.pdf`.) Expect JSON with the name "Asha Rao", skills, projects, `ats_score` and keywords.

```
powershell

python interview.py
```

Expect 5 questions for each of the 5 roles. Questions 2 and 3 should quote different resume bullets. The last line should say `Whisper ready: True`.

```
powershell

python gaze.py
python db.py
python shield.py
```

Expect `average -> 71`, then `db.py works`, then (Windows) `Windows API available: True` and `Nothing found`.

Test CopilotShield (Windows only)

Open a **third terminal**, go to `backend`, activate the venv, then:

```
powershell

python test_overlay.py
```

A purple window appears with `Capture exclusion applied: True (affinity now = 0x11)`. Take a screenshot with **Win + Shift + S**: the purple window should be missing or black in it.

Back in the first backend terminal:

```
powershell

python shield.py --watch
```

Within 2 seconds you should see a `hidden_overlay ... Window is hidden from screen sharing` line. Close the purple window: the next scan says `Nothing found`. Press **Ctrl + C** to stop.

If a normal app (password manager, video player) shows up while no copilot is running, add its exact process name in lowercase to `allow_processes` in `backend/data/known_tools.json`.

Test the frontend with fake data (no backend needed)

In the `frontend` terminal:

Windows PowerShell:

```powershell
$env:VITE_USE_MOCKS="true"; npm run dev
```

Linux/macOS:

```bash
VITE_USE_MOCKS=true npm run dev
```

Open http://localhost:5173 — the top-right badge says **Demo data**. Click through every page. Press **Ctrl + C** to stop. Close and reopen the terminal afterwards on Windows so the mock setting is cleared.

Test and tune eye tracking

With the app running (section 7), open http://localhost:5173/gaze-debug:

1. **Turn on camera**, then **Check my eye line (3 s)**.
2. **Start answer**. Look at the camera for 5 s, look away for 2 s, look back. Repeat 3 times.
3. **Stop answer**. Expect `look_away_count: 3` and `eye_contact_pct` roughly 60–75.

If looking straight at the camera shows red values in the **Difference** column, increase that tolerance in `GAZE_SETTINGS` at the top of `frontend/src/gaze/useGaze.js`. If looking clearly away stays green, decrease it.

---

7. Run CareerLens

Option A — one command

From the `careerlens` folder:

Windows (PowerShell or Command Prompt):

```powershell
.\start-careerlens.bat
```

Linux/macOS (first time only: `chmod +x start-careerlens.sh`):

```bash
./start-careerlens.sh
```

Two servers start and the browser opens http://localhost:5173 after about 10 seconds. On Windows, close the two "CareerLens Backend/Frontend" windows to stop. On Linux/macOS press **Ctrl + C** once.

Option B — two terminals (shows all logs clearly)

Terminal 1 — backend:

Windows PowerShell:

```powershell
cd careerlens\backend
venv\Scripts\activate
uvicorn main:app --port 8000
```

Linux/macOS:

```bash
cd careerlens/backend
source venv/bin/activate
uvicorn main:app --port 8000
```

Wait for `[main] Whisper speech model: ready`.

Terminal 2 — frontend:

```powershell
cd careerlens\frontend
npm run dev
```

(Linux/macOS: `cd careerlens/frontend` then `npm run dev`.)

Open **http://localhost:5173** in Chrome or Edge. To see and try the backend directly, open http://localhost:8000/docs.

To stop: press **Ctrl + C** in each terminal.

Do not add `--reload` to the uvicorn command for demos — it restarts the server and reloads Whisper.

---

8. Using the app

1. **Upload page** — choose the target role, drop the resume PDF, optionally paste a job description, click **Analyse my resume** (about 20 seconds). The three green dots

confirm the local AI is running.

2. **Resume report** — ATS score, checklist with tips, found and missing keywords, strengths, and rewritten bullets. Click **Start mock interview**.
3. **Interview room**
   - Tick the integrity-check box (needed for CopilotShield).
   - On the right: **Turn on camera**, then **Check my eye line (3 s)** while looking at the camera. Optional but gives eye-contact scores.
   - Click **Start interview**. For each question choose **Speak** (Start recording → Stop → Submit answer) or **Type**. Read the feedback, then **Next question**.
   - The shield light shows green "No AI assistants detected", red "Hidden assistant detected", or grey when not running.
4. **Final report** — 4 scores, integrity result, answer-by-answer feedback with a stronger answer, and the 2-week learning plan. **Print / save PDF** saves it.
5. **History** — every finished interview, with **Open report**.

A score shows **Not measured** when there was no data for it, for example Confidence when the camera was off and every answer was typed.

---

9. Demo day checklist

The day before

- ☐ All one-time downloads done on the demo laptop (section 5).
- ☐ **Turn Wi-Fi off** and run one full interview: upload → 5 answers (at least one spoken) → report. Everything must work offline.
- ☐ CopilotShield test passes with `test_overlay.py` (section 6).
- ☐ Eye tracking tuned on the demo laptop, in the demo room's lighting if possible.
- ☐ Backup demo video saved on two laptops and a pen drive.
- ☐ A clean sample resume PDF on the desktop.

30 minutes before

```powershell
cd careerlens
.\start-careerlens.bat
```

Then, in a new terminal (in `backend`, with `(venv)` active):

```powershell
```

```
ollama run qwen2.5:3b "hi"
python prewarm.py
```

`prewarm.py` loads the model into memory so the first question is fast, and prints the health of every part. Expect `ollama: True`, `whisper: True`, `shield: available`.

- [ ] Turn Wi-Fi off.
- [ ] Close other heavy apps (games, many browser tabs) to free memory.
- [ ] Browser allowed to use the camera and microphone for localhost.
- [ ] Open a terminal in `backend` with `(venv)` active, ready to run `python test_overlay.py`.

During the demo (about 5 minutes)

1. Show Wi-Fi is off: "Everything runs on this laptop."
2. Upload the resume → ATS score, missing keywords, rewritten bullets.
3. Eye-line check, then answer the first resume-claim question by voice.
4. Run `python test_overlay.py`: "This window is invisible to screen-share — the trick AI copilots use — but CareerLens sees it." The light turns red. Close it; the light turns green.
5. Answer one question by typing to show the fallback.
6. Final report: scores, stronger answer, learning plan, integrity result.

If anything fails, switch to the backup video immediately.

---

10. Settings you can change

What	Where	How
Use the smaller, faster model	`backend/llm.py`	change `MODEL = "qwen2.5:3b"` to `MODEL = "qwen2.5:1.5b"`, restart the backend
Fake data in the frontend	`frontend/.env`	copy `.env.example` to `.env`, set `VITE_USE_MOCKS=true`, restart `npm run dev`
Backend address	`frontend/.env`	`VITE_API_URL=http://localhost:8000`
Eye-tracking tolerances	`frontend/src/gaze/useGaze.js`	`GAZE_SETTINGS` at the top of the file
Copilot names / allow-list	`backend/data/known_tools.json`	edit `process_keywords` and `allow_processes` (lowercase)
Interview questions	`backend/data/question_bank.json`	5 technical questions per role, each with `key_points`
Role keywords	`backend/data/role_keywords.json`	role names must stay exactly: SDE, Data Analyst, ML Engineer, Embedded, Web Developer
Learning resources	`backend/data/resources.json`	only `https://` links are used
Delete all history	`backend/data/careerlens.db`	stop the backend and delete this file

Copy `.env.example` to `.env`:

```powershell
copy .env.example .env
```

(Linux/macOS: `cp .env.example .env`.)

11. API reference

Backend: http://localhost:8000 · interactive docs: http://localhost:8000/docs

Method	Endpoint	Sends	Returns
GET	`/api/health`	—	`ollama`, `whisper`, `whisper_status`, `shield`, module status
POST	`/api/resume/analyze`	form: `file` (PDF), `role`, `jd_text`	`resume_id`, `parsed`, `ats_score`, `ats_checks`, `keywords`, `feedback`
POST	`/api/interview/start`	JSON: `resume_id`, `role`, `consent`	`session_id`, first `question`
POST	`/api/interview/answer`	form: `session_id`, `question_id`, `audio` or `typed_text`, `gaze`	`transcript`, `input_mode`, `speech`, `gaze`, `evaluation`, `next_question`
POST	`/api/interview/finish`	JSON: `session_id`	`scores`, `score_notes`, `answers`, `roadmap`, `integrity`
GET	`/api/shield/status`	—	`status` (clear / alert / unavailable), `findings`, `active`, `reason`
GET	`/api/history`	—	list of past sessions with scores
GET	`/api/report/{session_id}`	—	a saved final report

Example response files are in `backend/mocks/`.

12. Troubleshooting

Problem	Fix
`python` is not recognised (Windows)	Reinstall Python and tick "Add python.exe to PATH", then reopen the terminal
`running scripts is disabled on this system`	Run `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`
Red "start Ollama" on the upload page	Start the Ollama app from the Start menu (Windows/macOS) or run `ollama serve` (Linux); check with `ollama list`
Every answer says "Not scored"	Ollama is not running or the model is missing: `ollama pull qwen2.5:3b`
Answers take more than 40 seconds	Switch to `qwen2.5:1.5b` in `backend/llm.py`, close heavy apps, run `prewarm.py` before the demo
"Cannot reach the CareerLens backend"	Start the backend (section 7) and check http://localhost:8000/api/health
`whisper_status: failed` in health	Run the Whisper download command in section 5 with internet on
Spoken answer gives "No answer was received"	Speak closer to the mic for at least 5 seconds, or type the answer
Microphone or camera blocked	Click the camera icon in the browser address bar → Allow → reload the page
Camera starts but eye tracking never works	Check `public/mediapipe/` files exist (section 4)
Eye contact always low or always 100%	Redo "Check my eye line", face the light, sit 50–70 cm away, tune `GAZE_SETTINGS`
Shield light always grey "unavailable"	The integrity box was not ticked, or the laptop is not Windows 10 (2004+) / 11
Shield light red with no copilot running	Add that app's process name to `allow_processes` in `known_tools.json`
`test_overlay.py` says capture exclusion failed	Windows is older than version 2004 — update Windows

Problem	Fix
`Port 8000 is already in use` (Windows)	`netstat -ano | findstr :8000` then `taskkill /PID <number> /F`
`Port 8000 is already in use` (Linux/macOS)	`lsof -i :8000` then `kill <PID>`
`Port 5173 is in use`	Close the other `npm run dev` window, or use the port commands above with 5173
`pip install` fails on faster-whisper	Use Python 3.11, then repeat step 3.3 in a fresh `venv`
Resume shows "No readable text was found"	The PDF is a scan or photo. Export the resume from Word/Google Docs as PDF
Backend prints `[main] WARNING: could not import ...`	The named file is missing or has an error — the message shows the reason

---

13. Team ownership

Member	Area	Files
1	Frontend	`frontend/src` (except `gaze/`)
2	Resume, ATS, scoring, roadmap	`backend/resume.py`, `backend/scoring.py`, `data/role_keywords.json`, `data/resources.json`
3	Interview engine, Whisper	`backend/interview.py`, `data/question_bank.json`
4	GazeGuard-lite, slides, testing	`frontend/src/gaze/`, `backend/gaze.py`
5	CopilotShield-lite, integration	`backend/main.py`, `llm.py`, `db.py`, `shield.py`, `test_overlay.py`, `prewarm.py`, `data/known_tools.json`

Only Member 5 edits `main.py`. See `CHANGES.md` for the fixes made during integration.

---

14. Privacy

- **No internet use** after the one-time downloads. The language model, speech recognition and face tracking all run on the laptop.

- **Video never leaves the browser.** Only numbers (eye-contact %, look-away count) are sent to the local backend.
- **Audio is deleted** right after it is transcribed.
- **Face detection, not recognition.** CareerLens measures where the eyes point; it never identifies who a person is.
- **CopilotShield is opt-in** and runs only during an interview. It reads window flags and process names only, never screen contents, files or browsing.
- **Integrity results are evidence for review, never an automatic verdict.**
- History is stored only in `backend/data/careerlens.db` on the laptop.